

UFPE – CENTRO DE INFORMÁTICA
ESTRUTURAS DE DADOS ORIENTADAS A OBJETOS
ATIVIDADE PRÁTICA

(Gustavo Carvalho – ghpc@cin.ufpe.br)

Exercícios selecionados de: KIRCH-PRINZ, U., PRINZ, P.
A Complete Guide to Programming in C++.
1a Edição. Editora Jones & Bartlett Learning, 2001.

Exercise 1

Write a C++ program to initialize a string `s1` with the string "As time by ..." and a second string `s2` with the string "goes",

- insert string `s2` in front of "by" in string `s1`,
- erase the remainder of string `s1` after the substring "by",
- replace the substring "time" in `s1` with "Bill".

In each case, your program should determine the position of the substring. Output string `s1` on screen at the beginning of the program and after every modification

Exercise 2

Write a C++ program that reads a word from the keyboard, stores it in a string, and checks whether the word is a palindrome. A palindrome reads the same from left to right as from right to left. The following are examples of palindromes: "OTTO", "deed", and "level." Use the subscript operator `[]`. Modify the program to continually read and check words.

Exercise 3

Write a function to calculate the factorial of a number.

- Argument: A number `n` of type unsigned int.
- Returns: The factorial `n!` of type long double.

Formulate two versions of the function, where the factorial is

- calculated using a loop
- calculated recursively

Test both functions by outputting the factorials of the numbers 0 to 20.

Exercise 4

In general, you should use different names for different objects. However, if you define a name for an object within a code block and the name is also valid for another object, you will reference only the new object within the code block. The new declaration hides any object using the same

name outside of the block. When you leave the code block, the original object once more becomes visible. The program below uses identical variable names in different blocks. What does the program output on screen? Run the program to test your guess.

```
// scope.cpp
// Accessing objects with equal names
// -----
#include <iostream>
#include <iomanip>
using namespace std;
int var = 0;
namespace Special { int var = 100; }
int main() {
    int var = 10;
    cout << setw(10) << var;
    {
        int var = 20;
        cout << setw(10) << var << endl;
        {
            ++var;
            cout << setw(10) << var;
            cout << setw(10) << ++ ::var;
            cout << setw(10) << Special::var * 2 << endl;
        }
        cout << setw(10) << var - ::var;
    }
    cout << setw(10) << var << endl;
    return 0;
}
```

Exercise 5

You are developing a large-scale program and intend to use two commercial libraries, `tool1` and `tool2`. The names of types, functions, macros, and so on are declared in the header files `tool1.h` and `tool2.h` for users of these libraries. Unfortunately, the libraries use the same global names in part. In order to use both libraries, you will need to define namespaces. Write the following program to simulate this situation:

- Define an inline function called `calculate()` that returns the sum of two numbers for the header file `tool1.h`. The function interface is as follows:
`double calculate(double num1, double num2);`
- Define an inline function called `calculate()` that returns the product of two numbers for a second header file `tool2.h`. This function has the same interface as the function in `tool1.h`.

- Then write a source file containing a main function that calls both functions with test values and outputs the results.

To resolve potential naming conflicts, define the namespaces `TOOL1` and `TOOL2` that include the relevant header files.

Exercise 6

Test your knowledge of external and static variables by considering the following program. What screen output does the program generate? Run the program to test your guess.

```
// static.cpp
// Tests an internal static variable
// -----
#include <iostream>
#include <iomanip>
using namespace std;
double x = 0.5,
fun(void);

int main() {
    while(x < 10.0){
        x += fun();
        cout << "\tWithin main(): "
              << setw(5) << x << endl;
    }
    return 0;
}

double fun() {
    static double x = 0;
    cout << "Within fun(): " << setw(5) << x++;
    return x;
}
```

Exercise 7

The version of the function `swap()` presented below can be compiled without producing any error messages. However, the function will not swap the values of `x` and `y` when `swap(&x, &y);` is called. Correct the program such that it works as expected.

```
#include <iostream>
using namespace std;
```

```
void swap(float *p1, float *p2) {
    float *temp;
    temp = p1;
    p1 = p2;
    p2 = temp;
}

int main() {
    float x = 2.1;
    float y = 3.2;

    swap(&x, &y);
    cout << "x = " << x << ", y = " << y << endl;

    return 0;
}
```